Windows Lockdown Operator's Manual

(hardener_windows.ps1)

⚠️ Golden Rules Before You Touch Anything ⚠️

1. **Perform Pre-Execution Sweep:** Complete your manual persistence triage (`quser` for active sessions, `lusrmgr.msc` for local admins, `schtasks` for persistence) **before** executing this script. "*Refer to SOP Manual.*"

2. **Run Locally:** This script runs **locally** on whatever Windows machine you are currently sitting at. `$TargetServerIP` is for log files and messages only; it does **not** magically execute against a remote box.

3. **Keep Out-of-Band Access Open:** Always ensure you have hypervisor console access (Proxmox / VMware VMRC / Proxmox VNC) open in case you need emergency recovery. If a firewall rule cuts you off, you need that console to get back in.

4. **Verify with a Teammate:** Do not fill in ports or IPs alone under panic. Have a teammate double-check your numbers before flipping the safety switch.

Step-by-Step Execution Guide

Step 1: Pre-Execution Safety Check & Opening PowerShell

- **Open Out-of-Band Console:** Confirm hypervisor console access is active.

- **Open PowerShell correctly:**
  - Press the **Windows Key**, type `powershell`.
  - **Right-click Windows PowerShell** and select **Run as administrator**.

- **Beginner Trap:** If you don't run as Administrator, the script will instantly throw an error and quit. That is normal and intentional!

Step 2: Fill in Your Topology Details

Open your script file (`hardener_windows.ps1`) in VS Code or Notepad and fill in the values based on your team's map packet:

1. **Target Server IP:**

```
$TargetServerIP = "10.1.1.20" # Put YOUR machine's IP here
```

2. **Scored Ports:** Verify or add any application ports required to keep points green (e.g., HTTP `80`, HTTPS `443`, RDP `3389`, or database/DNS ports):

```
$ScoredPorts = @(
    [pscustomobject]@{ Protocol = "TCP"; Port = 80 },   # Web (IIS/Apache)
    [pscustomobject]@{ Protocol = "TCP"; Port = 443 },  # Secure Web
    [pscustomobject]@{ Protocol = "TCP"; Port = 3389 }  # RDP
)
```

3. **Unscored Services:** Leave default risky services (`RemoteRegistry`, `WMPNetworkSvc`, `SNMP`) to be killed, or add any newly discovered ones.

Step 3: Lift the Safety Gate (Crucial)

Near the top of the script, find the safety switch and change it from `$false` to `$true`:

```
$TopologyReviewed = $true
```

- **Beginner Trap:** If you leave this as `$false`, the script will refuse to run and throw an error. This is a built-in safety check so you don't run default placeholders.

Step 4: Execute the Script & Input Password

1. Navigate to the folder where your script is saved in PowerShell:

```
cd C:\path\to\script
```

Note: "`C:\path\to\script`" is a placeholder. Replace it with the actual folder path where you saved `hardener_windows.ps1` on your machine.

2. Run the script:

```
.\hardener_windows.ps1
```

Note: If PowerShell blocks script execution, run this exact command to bypass restrictions for your current process and run the script.

```
Set-ExecutionPolicy Bypass -Scope Process; .\hardener_windows.ps1
```

3. When prompted on screen, type your team's **Master Administrator Password** securely.

Step 5: Post-Execution Verification Checklist

1. **Check the Scoring Dashboard:** Immediately look at the scoreboard/UI to ensure your application ports and services stay **Green**.

2. **Test Your Login:** Open a new PowerShell or RDP session and log in with your new team password to verify you didn't lock yourself out.

3. **Review the Transcript Log:** If anything went wrong or feels broken, check the automatically generated audit transcript log located at `C:\ProgramData\CCDC\Logs\`.

🛑 Common Beginner Error Decoder

If the script stops or throws red text, don't panic. Look at this quick decoder:

Error / Warning Message	What Happened?	How to Fix It
`Set TopologyReviewed to $true...`	You forgot to flip the safety switch in Step 3.	Open the script, change `$TopologyReviewed = $true`, save, and re-run.
`This script must be run with Administrator privileges!`	You opened a standard PowerShell prompt instead of an elevated one.	Close the window, right-click PowerShell, and select **Run as administrator**. PNG
`Rule '...' exists; review its port...`	A firewall rule with that name was already configured on the box.	This is just a warning. The script skips creating a duplicate and keeps moving safely.
`Expected one built-in local Administrator account...`	The machine's built-in Admin account (SID `-500`) was renamed or disabled by the base image.	Check local users (`lusrmgr.msc`) to see what the active admin account is, though SID `-500` should resolve on standard Windows builds.

Summary

This Script is designed to be a Windows hardening script for CCDC competitions that functions as an automated tool to rapidly lock down systems during high stress environments. Do not deploy this script before manually sweeping the network and conducting an eviction process. Running this automated script blindly will rotate passwords and enable firewalls, and if you do not purge active threats, red-teams persistence stays intact. Furthermore, Attackers could possibly maintain hidden footholds through pre-staged backdoors implemented prior to competition starting possible rogue service accounts, or even malicious scheduled tasks, allowing red teams to compromise the system all over again even after administrative access is restricted, rendering this script completely redundant.

Following the manual eviction sweep and blue teams clear out active compromises; this script should be deployed. This script initiates its sequence by performing safety validations as guardrails to ensure deployment integrity. It verifies that blue teams are running the session with admin privilege. This script will also prevent execution unless the blue team manually changes the topology Boolean check to true, ensuring an extra layer of safety to prevent deployment without the correct topology information. This ensures configuration for second ports and services do not contain conflicting entries or invalid port numbers, do not switch safety off without double checking topology map. Following these checks, the script establishes secure audit trail by initializing a timestamp transcript log inside the ProgramData directory, ensuring a built-in safety net that ensures the log is properly saved and closed, even if the script crashes or gets interrupted halfway through deployment.

Hardening phase is carried out in a controlled order once deployed. Phase 1, it targets the built-in local administrators account specifically designed to look for standard ending identifiers to guarantee the password is successfully rotated regardless of whether the acount name was altered. Phase 2, it enables the Windows Firewalls across all network profiles and iterates through the defined configuration to create a clean inbound allow rules for essential scored ports, with built in optional capabilities to restrict all incoming traffic except the official scoring engine addresses if it's provided by the topology map. Phase 3, this script loops through the specified list of all unscored background services, and brute force stops them and changes their startup type to disabled to eliminate known attack vectors and reduce the system attack surface.

Finally, this script supports optional advanced hardening toggles built in that can be enabled based on team strategy. These include evicting unauthorized remote desktop sessions by reading active user tables and terminating unwanted session identifiers while in turn protecting blue team operational sessions. This can also safely restart remote management services under local execution safeguards as well as apply baseline registry security configurations to disable multicast name resolution (LLMNR) and restrict anonymous share or SAM enumeration and then enforce cryptographic signing for Server Message Block communications. Through this custom sequence, the script systematically transforms a vulnerable base image into a locked-down and auditable host while ensuring critical services stay intact before red teams can make any real progress.

(hardener_windows.ps1)

```
<#
.SYNOPSIS
    Rapid Windows Hardening Script for CCDC with Topology Placeholders
.DESCRIPTION
    Rotates local admin password, safely appends firewall rules for scored ports,
    disables unscored background services, and logs all changes to an audit
transcript.
#>

# ================================================================================
# TOPOLOGY & CONFIGURATION PLACEHOLDERS (FILL ME IN WHEN MAP DROPS)
# ================================================================================

# 1. SAFETY GATE: Leave this as $false until you have filled in your topology
details.
# The script will refuse to run if this is false, preventing accidental default
runs.
$TopologyReviewed =$false

# 2. TARGET SERVER IP: Enter the IP address assigned to THIS specific machine.
$TargetServerIP = "[PUT_SERVER_IP_HERE]"         # e.g., "10.1.1.20"

# 3. SCORING ENGINE IPs: Enter the IP address(es) of the official scoring engine.
$ScoringSourceIPs = @(
    # "[PUT_SCORING_ENGINE_IP_HERE]"             # e.g., "10.1.1.50"
)

# 4. SCORED PORTS: Add every TCP/UDP port that your services require to stay up
for points.
$ScoredPorts = @(
    [pscustomobject]@{ Protocol = "TCP"; Port = 80 },   # HTTP (Web)
    [pscustomobject]@{ Protocol = "TCP"; Port = 443 },  # HTTPS (Secure Web)
    [pscustomobject]@{ Protocol = "TCP"; Port = 3389 }  # RDP (Remote Management
- verify policy)
    # TODO: Add any server-specific scored ports here (e.g., 53 for DNS, 1433 for
MSSQL, 3306 for MySQL)
)

# 5. SCORED SERVICES: List any core application services that must NEVER be
stopped.
```

```powershell
$ScoredServices = @(
    # "<SERVICE_NAME>"                          # e.g., "W3SVC" for IIS,
"MSSQLSERVER" for SQL
)

# 6. UNSCORED SERVICES: List vulnerable or unnecessary background services to
kill and disable.
$UnscoredServices = @(
    "RemoteRegistry",
    "WMPNetworkSvc",
    "SNMP"
    # TODO: Add any other non-scored or vulnerable background services discovered
during analysis
)

# 7. OPTIONAL HARDENING TOGGLES: Flip these to $true if your team decides to
enforce them.$Options = @{
    RestrictPortsToScoringSources = $false  # Lock inbound traffic strictly to
Scoring Engine IPs
    EvictSelectedRdpSessions      = $false  # Log off unauthorized active RDP
sessions
    RestartWinRM                  = $false  # Restart WinRM (Keep false if
running remotely!)
    EnforceSmbSigning             = $false  # Enforce SMB signing (Verify legacy
client compatibility first)
}
# ============================================================================

# Ensure script is run with administrator privileges
If (-not
([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCu
rrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)) {
    Write-Error "[-] This script must be run with Administrator privileges!
Right-click PowerShell and select Run as administrator."
    Exit 1
}

# Enforce Topology Safety Gate Check
if (-not $TopologyReviewed -or $TargetServerIP -match '^<.*>$') {
    throw "[-] ABORT: You must change `$TopologyReviewed to `$true and fill in
your `$TargetServerIP before running this script!"
}
```

```powershell
# Check for accidental conflicts between scored and unscored service lists
$Conflicts = @($UnscoredServices | Where-Object { $_ -in $ScoredServices })
if ($Conflicts.Count -gt 0) {
    throw "[-] ERROR: A service is listed as both scored and unscored:
$($Conflicts -join ', ')"
}

# Validate scoring sources if port restriction is enabled
if ($Options.RestrictPortsToScoringSources -and -not $ScoringSourceIPs.Count) {
    throw "[-] ERROR: Port source restriction is enabled, but no scoring source
IPs are listed in `$ScoringSourceIPs."
}

# Validate structure of scored ports
foreach ($Spec in$ScoredPorts) {
    if ($Spec.Protocol -notin @("TCP", "UDP") -or $Spec.Port -notin 1..65535) {
        throw "[-] ERROR: Invalid port entry: $($Spec | Out-String)"
    }
}

# Optional plaintext password for rapid deployment. Leave blank to be prompted
securely on screen.
$NewPassword = ""

# ================================================================================
# EXECUTION & TRANSCRIPT LOGGING
# ================================================================================
$TranscriptStarted =$false
$LogDirectory = Join-Path$env:ProgramData "CCDC\Logs"
New-Item -ItemType Directory -Path $LogDirectory -Force | Out-Null
$LogPath = Join-Path$LogDirectory "Hardening-$env:COMPUTERNAME-$(Get-Date -Format
'yyyyMMdd-HHmmss').log"

try {
    try {
        Start-Transcript -Path $LogPath -ErrorAction Stop | Out-Null
        $TranscriptStarted =$true
    }
    catch {
        Write-Warning "[-] Warning: Transcript logging could not start: $_"
    }
```

```
    Write-Host "[+] Starting Windows Hardening and Lockdown for server:
$TargetServerIP" -ForegroundColor Green

    # STEP 1: Rotate the built-in local Administrator password securely (Targets
SID -500)
    if ([string]::IsNullOrWhiteSpace($NewPassword)) {$SecurePassword = Read-Host
"Enter new team master password for the local Administrator account" -
AsSecureString
    }
    else {
        $SecurePassword = ConvertTo-SecureString$NewPassword -AsPlainText -Force
    }

    $AdminAccounts = @(
        Get-LocalUser -ErrorAction Stop |
            Where-Object { $_.SID.Value -match '-500$' }
    )

    if ($AdminAccounts.Count -ne 1) {
        throw "Expected one built-in local Administrator account; found
$($AdminAccounts.Count)."
    }

    Set-LocalUser -Name $AdminAccounts[0].Name -Password$SecurePassword -
ErrorAction Stop
    Write-Host "[+] Local Administrator password successfully updated." -
ForegroundColor Cyan

    # STEP 2: Enable Windows Firewall & Open Scored Ports Safely
    Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled True
    Write-Host "[+] Windows Firewall enabled for all network profiles." -
ForegroundColor Cyan

    foreach ($Spec in $ScoredPorts) {$RuleName = "CCDC-Allow-$($Spec.Protocol)-
$($Spec.Port)"
        if (Get-NetFirewallRule -DisplayName $RuleName -ErrorAction
SilentlyContinue) {
            Write-Warning "Rule '$RuleName' already exists. Skipping recreation."
            continue
        }

        $Rule = @{
            DisplayName = $RuleName
```

```
            Direction   = "Inbound"
            Protocol    = $Spec.Protocol
            LocalPort   = $Spec.Port
            Action      = "Allow"
            ErrorAction = "Stop"
        }

        if ($Options.RestrictPortsToScoringSources) {
            $Rule.RemoteAddress =$ScoringSourceIPs
        }

        New-NetFirewallRule @Rule
        Write-Host "[+] Allowed inbound traffic on port: $($Spec.Port)
($($Spec.Protocol))" -ForegroundColor Green
    }

    # STEP 3: Disable Risky and Unscored Background Services
    foreach ($Service in$UnscoredServices) {
        if ($Service -ne "[PUT_SERVICE_NAME_HERE]" -and (Get-Service -Name
$Service -ErrorAction SilentlyContinue)) {
            Stop-Service -Name $Service -Force -ErrorAction SilentlyContinue
            Set-Service -Name $Service -StartupType Disabled
            Write-Host "[+] Disabled and stopped unneeded service: $Service" -
ForegroundColor Yellow
        }
    }

    # STEP 4: Evict Unauthorized RDP Sessions (Optional Opt-In)
    if ($Options.EvictSelectedRdpSessions) {
        quser.exe
        $RdpSessionIdsToLogoff = @(
            # Type authorized session IDs here if you need to kick intruders out
        )
        $CurrentSessionId = (Get-Process -Id$PID).SessionId

        foreach ($SessionId in$RdpSessionIdsToLogoff) {
            if ($SessionId -eq$CurrentSessionId) {
                Write-Warning "Skipping your current active session ID
($SessionId)."
                continue
            }

            & "$env:SystemRoot\System32\logoff.exe" $SessionId
```

```
            if ($LASTEXITCODE -ne 0) {
                Write-Warning "Failed to log off RDP session ID $SessionId."
            }
        }
    }

    # STEP 5: Restart WinRM Service (Opt-In Out-of-Band Safeguard)
    if ($Options.RestartWinRM) {
        if ($PSSenderInfo) {
            throw "Refusing to restart WinRM from a remote PowerShell session to
prevent self-lockout."
        }
        Restart-Service -Name WinRM -Force -ErrorAction Stop
    }

    # STEP 6: Apply Low-Risk Baseline Hardening (LLMNR & Anonymous Shares)
    Write-Host "[+] Applying low-risk baseline security controls..." -
ForegroundColor Cyan
    $DnsClientPolicy = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient"
    New-Item -Path $DnsClientPolicy -Force | Out-Null
    New-ItemProperty -Path $DnsClientPolicy -Name EnableMulticast -PropertyType
DWord -Value 0 -Force | Out-Null

    $LsaPolicy = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
    New-ItemProperty -Path $LsaPolicy -Name RestrictAnonymous -PropertyType DWord
-Value 1 -Force | Out-Null
    New-ItemProperty -Path $LsaPolicy -Name RestrictAnonymousSAM -PropertyType
DWord -Value 1 -Force | Out-Null
    New-ItemProperty -Path $LsaPolicy -Name EveryoneIncludesAnonymous -
PropertyType DWord -Value 0 -Force | Out-Null

    # STEP 7: SMB Signing (Optional Opt-In)
    if ($Options.EnforceSmbSigning) {
        Set-SmbServerConfiguration -RequireSecuritySignature $true -Force
        Set-SmbClientConfiguration -RequireSecuritySignature $true -Force
    }

    Write-Host "[+] Windows lockdown completed successfully for $TargetServerIP!"
-ForegroundColor Green

}
catch {
    Write-Error "[-] Critical error encountered during script execution: $_"
```

```
    throw
}
finally {
    if ($TranscriptStarted) {
        Stop-Transcript | Out-Null
        Write-Host "[+] Execution audit transcript log saved securely to:
$LogPath" -ForegroundColor Cyan
    }
}
```



Written By Sauce

10/18/2026

